

CONCEPT NOTE

APIC ProtoHub

Prototyping-as-a-Service for Andhra Pradesh

Andhra Pradesh has built the machines. It has not built the way in. ProtoHub turns scattered prototyping centres into one bookable network — so idle machines find innovators, and ideas stop dying for lack of access.

The problem

An APIC is a workshop of high-value equipment — 3D printers, CNC mills, laser cutters, PCB fabrication. The State has funded several across Andhra Pradesh. Each one is well equipped and badly connected.

A student in Kakinada with a design cannot find out which centre exists, what machines it holds, whether any of them can make her part, when one is free, or what it will cost. She asks someone who asks someone. Meanwhile the centre manager has crores of capex idle and no channel to reach her. The Innovation Society, which paid for all of it, cannot answer the simplest question: which machines are being used?

Nobody is at fault. The assets exist; the operating system does not.

WHAT BREAKS TODAY

- **Zero demand visibility.** Machines idle not from lack of demand, but because nobody knows they exist.
- **Manual scheduling and quoting.** Bookings live in registers and staff memory; pricing depends on who you ask. Double-bookings and no-shows are structural, not accidental.
- **Islands, not a network.** Kakinada's printer breaks and nobody routes the job to Vijayawada. Capability is duplicated in some districts and absent in others.
- **No utilization data, reactive inventory.** Managers cannot justify equipment; APIS cannot measure return on capex; filament runs out mid-job.

The solution

ProtoHub is one digital layer over every APIC. Innovators describe a part rather than choose a machine; the platform returns only equipment that can physically make it, prices the job, holds a slot, and tracks it to collection. Staff run a live queue instead of a register. APIS sees utilization across the network for the first time.

FIT — THE CAPABILITY ENGINE

The core insight: users know their part, not our equipment list. Every other feature assumes you already know which machine to book. Fit removes that assumption.

Given size, material, and precision, Fit filters the network by build envelope, achievable tolerance, minimum feature resolution, and live material stock — then ranks by availability, distance, and fit, so a small part never consumes a high-value machine.

CAPABILITY	HOW PROTOHUB DELIVERS IT
Discover	Describe the part, not the machine. Rotation-aware envelope match, tolerance and min-feature filters, live stock. Unlisted centres never surface.
Book	Live slots from centre hours and existing bookings. Double-booking is rejected by a database exclusion constraint — not by application code. No partial slots past closing.
Quote	Versioned and line-itemised: machine time, material grams, setup, student discount. Staff-priced today; every manual quote trains the engine that automates it.
Track & collaborate	Explicit state machine: queued → setup → running → QC → ready → collected, each transition logged with actor and timestamp. Job threads carry models, drawings, and staff notes — expertise reaches the innovator before a bad design wastes material.
Analyse	The tracking log is the utilization dataset. Booked hours per machine per week and no-show rates, segmented by district, process, and user type.
Manage	Per-machine stock with reorder thresholds flags low material before a job fails. Maintenance is booked like any job — scheduled and logged, never improvised.

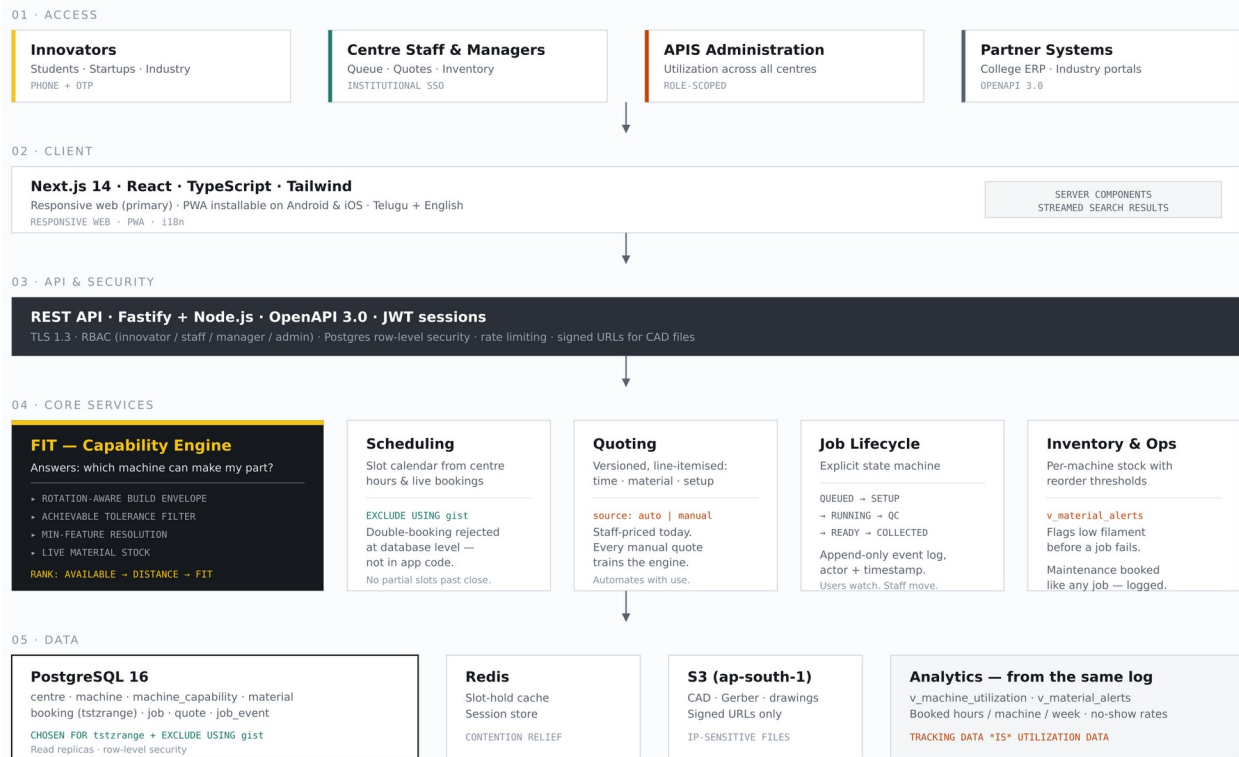
WHAT IS ALREADY BUILT AND TESTED

This is not a concept awaiting a prototype. The data model, capability engine, and booking flow run against a live PostgreSQL 16 instance and are verified by test:

- ✓ Rotation-aware fit — a 320×330×500mm part correctly matches a 340×340×600mm machine
- ✓ Double-booking rejected at database level; cancelled bookings correctly free the slot
- ✓ No 1-hour slot offered at 17:00 when the centre closes 17:30 — but a 30-min slot is
- ✓ Uncommissioned centres never surface in search
- ✓ Low-stock machines flagged before a job is accepted

The matcher was implemented twice — in SQL and in JavaScript — and both return identical results across six scenarios. Agreement between independent implementations is what distinguishes correct logic from logic that happens to pass.

Architecture



Scale and expansion

- **Capability is data, not code.** A new machine type needs no release — a waterjet and an SLS printer are the same row shape. Adding a centre is an INSERT; Anantapur goes live by flipping one boolean.
- **The network outgrows APICs.** The schema hardcodes no "APIC" — a centre is any facility with machines. College workshops, ITI labs, and industry spare capacity join the same network. Public capex is the seed, not the ceiling.
- **The platform improves with use.** Every quote records whether it was priced automatically or by hand. Each manual quote is a labelled training example, so the engine gets cheaper to run the more the network is used.
- **Horizontal by default, interoperable from day one.** Stateless containers scale on request volume; Postgres reads scale via replicas. OpenAPI 3.0 lets APIS dashboards, college ERPs, and partner portals integrate without bespoke work. IoT telemetry is additive — a sensor writes machine status where a technician does today.

The honest constraint

Technical scale is solved. Organisational scale is not. This platform succeeds or fails on whether a centre manager who prefers his register chooses to open it — so every feature must make his day easier on the first morning, not only the innovator's. Likewise, our prototype's seed data is our own: machine specifications are accurate for the models named, but rates and centre details are estimates. Before a pilot, one real centre's machine list and price card is required — a phone call that will change the schema in small ways no design can predict from outside.